OrchLang: Replayable Contracts for Token Usage and Request-Equivalent LLM Workflows

Nambi Rajan M*

Affiliation and postal address to be confirmed by the author

Abstract

Equal token budgets do not establish that a language-model workflow's usage is independent of private routing. OrchLang is offline research software that separates bounded accounting, request equivalence, and model-specific usage distributions. Its C++ compiler resolves typed requests and dependent results; a graph checker supports productive cycles and independently replayed product certificates. Rational-kernel tools quantify finite usage leakage, check sufficient stochastic bisimulation, and decide ordered trace-law equality at every finite horizon by exact linear algebra. Equality certificates and finite inequality witnesses are independently replayed. Narrow source bridges connect admitted OrchLang and Python LangGraph programs to replay-gated execution. Fifteen balanced source projections are admitted, versus eight by the preceding compiler. Independent bounded semantics find no accepted difference in 13,824 secret comparisons; exact channel analysis agrees with independent enumeration in 260 comparisons. An authored equal-law example resolves a bisimulation-inconclusive case. The contribution specializes established relational and probabilistic methods in an inspectable software pipeline. Guarantees require declared accounting and successful-outcome assumptions; provider invoices, unrestricted framework translation, termination and whole-cycle budgets are outside its scope.

Keywords: static analysis, confidentiality, token accounting, probabilistic verification, research software, reproducibility

1. Motivation and significance

A private flag may select one model call on either of two public inputs bounded by the same token cap. Equal worst-case budgets do not prevent

*Correspondence details await author confirmation. Research draft, 9 October 2026; not submitted.

their concrete input lengths from revealing the flag. Conversely, different requests can have the same usage law under a particular stochastic model. These distinctions matter when a per-run observer sees usage records and knows the public context.

Constant-resource typing and relational cost analysis already study information flow through resource use [1, 2]. PPDL models uncertain flows with multiple language-model calls and uses token budgeting [3]; quantitative information flow supplies vulnerability and entropy measures [4, 5]. OrchLang specializes these established methods. Its software retains resolved model identity, exact typed requests, corresponding earlier outcomes, source provenance and independently checked evidence for a declared usage observer. Researchers can compare conservative checks valid for any admitted outcome oracle with precise examples under one stochastic model. External adoption and practical impact are unmeasured.

2. Software description

2.1. Software architecture

The C++17 compiler performs lexing, parsing, scoped resolution, flow-policy checking, cost analysis, relational analysis, dependency-graph generation, certificate emission and offline execution. The bounded source language supports calls, branches, literal-bounded retries, outputs, effects and explicit releases; it has no unbounded loop or concurrency. Sequence adds token-bound components, branches take componentwise maxima and retries multiply bounds. Saturation and invalid runtime inputs fail closed.

A separate Python graph frontend handles ordered calls, mutable result registers, scalar guards and productive cycles. Its product state (pc_L, pc_R, R) tracks two control positions and a sufficient equal-value relation. Secret choices generate universal obligations; aligned public guards require equal subjects and predicates. Calls require equal resolved contracts, templates and ordered typed literals or related values. Assignment removes stale result relations before adding the new pair. Finite closure closes repeated product states; stops must match and reachable call-free cycles reject. A separately implemented replayer reconstructs the complete rooted closure without importing the producer.

Two narrow adapters export bounded OrchLang source and static Python LangGraph source. Contracts supply input labels, bounds and model declarations; the Python AST supplies calls and control. Unsupported effects, dynamic syntax, reducers and parallel edges reject. The AST adapter checks LangGraph's atomic returned-state updates before lowering them to sequential registers. An actual pinned LangGraph backend requires independent

certificate replay and executes bounded offline sessions. Immutable source and compiler snapshots bind translation evidence through recorded digests. The translation guarantee is limited to the documented subsets and finite differential checks.

2.2. Software functionalities and assumptions

The resolved contract contains provider, model identifier, output cap and optional nominal binary64 price metadata. The abstract request contains exact template bytes and ordered typed atoms or full argument values. Provider envelopes, tokenizers, hidden system messages, caches and account state are absent. Bounds require compatible argument units, cap-respecting outputs and full modeled input usage bounded by the declared additive UTF-8 estimates. Price metadata is not an exact currency theorem.

A shared complete-outcome oracle keyed by contract, per-contract ordinal and request gives corresponding equal outcomes; retries also require coupled success events. Under equal public inputs, admitted bounded workflows without declassification have equal modeled request and usage traces. Productive graph cycles have matching finite prefixes and stop positions; they need not terminate or have a finite total budget. Explicit-data labels permit dependent calls under matched private control, while effective labels still enforce output/effect policies. Declared releases withhold the unrestricted secrecy guarantee.

For model-specific analysis, users declare a finite rational kernel indexed by contract, ordinal, exact request and the last zero to four complete outcome events. Exact dynamic programming retains all future-relevant values, control, ordinals, observation and history. Identical states merge probability mass. A completed histogram must have mass exactly one; caps produce unknown without partial-safe conclusions. The tool reports exact total variation and Bayes vulnerability, and symbolic mutual information with approximate logarithm displays. The channel covers selected secret assignments, a fixed public store, successful calls and integer usage; it excludes errors and real-provider probability fitting.

An additional finite semantic transducer omits accumulated usage and history-independent work counters. It saturates ordinals at 33: every literal ordinal/history test in kernel version 1 is at most 32. Finite outcome support, fixed inputs and bounded complete history therefore give a finite reachable state system. Exact partition refinement groups states with the same terminal class and distribution over visible usage labels and successor blocks, following established probabilistic bisimulation [6]. Equal initial blocks imply equal ordered success-only usage laws at every finite call horizon. Split

blocks are inconclusive, not leakage witnesses. This result excludes cumulative runtime resource monitors and any infinite-run or total-cost claim.

For a stable partition $\mathcal{P}$, each pair s, t in a block satisfies, for every visible label ℓ and target block B,

$$\sum_{u \in B} P(s, \ell, u) = \sum_{u \in B} P(t, \ell, u). \tag{1}$$

Terminal status also agrees. Induction on the remaining horizon gives equal trace laws. The independent replayer reconstructs transitions and checks this equation using exact rational arithmetic.

Because strong bisimulation is incomplete for observable-law equality, an exact trace-equivalence tool applies established weighted-automaton span reasoning [7, 8, 9]. APEX previously used program-to-automata equivalence for anonymity and other probabilistic properties [10]. For each selected secret scenario i against scenario 0, let $\mu_i = e_i - e_0$, A_ℓ be the rational transition matrix for a visible usage label, and b_S, b_N indicate stopped and nonterminal states. Equality holds exactly when

$$\mu_i A_w b_S = \mu_i A_w b_N = 0 \quad \text{for every finite word } w. \tag{2}$$

The implementation explores a rational basis for reachable rows $\mu_i A_w$. A closed basis annihilating both columns certifies equality; a nonzero output gives a finite word with exact differing probabilities. Independent replay reconstructs the system, word rows, span closure and both outputs. Work, matrix-entry and precision caps give unknown without a partial decision. Completeness concerns the supplied finite model and listed inputs; provider premises and infinite-run properties remain excluded.

2.3. Installation and use

Core compilation needs `make` and C++17; the audit tools use Python 3.10+ and the standard library. Checking and replay are available in a wheel tested from an isolated directory. LangGraph execution is optional, pinned to version 1.2.14 with a dependency lock [11]. No provider key, paid call or real tool execution is needed.

```
make check
./orchc check \
  examples/research/accepted/chain_matched.orch
python3 audit/orchgraph.py GRAPH --report CERTIFICATE
python3 audit/replay_graph_certificate.py GRAPH CERTIFICATE
python3 audit/reproduce_final.py --sanitize --package
```

The reproduction manifest records code/result hashes, commands, outcomes and environment. Optional integration execution requires its pinned environment and is recorded separately. Documentation specifies each subset and theorem boundary, negative cases, provenance and unknown outcomes. A closing HTML demonstration runs the real local tools, with any saved-evidence fallback explicitly labeled. AI-assisted development and adversarial review used Codex and GPT-6 Luna; tools also assisted literature checking, test generation and drafting. The human author must review implementation, sources and disclosures before submission.

3. Illustrative examples

Matched private alternatives can each call a model on the same public query and then on the corresponding earlier result. The first requests agree; coupled outcomes agree; the second requests agree. This now passes, while changing the template, model contract or outer argument fails. Boolean `true` and text `"true"` are distinct request atoms; an audit found and repaired their earlier collision.

A quantitative example gives output usage 1 or 3 with equal probability in one arm, and deterministic usage 2 in the other. Both means are 2, but total variation is 1: the observation identifies the private arm. Another emits $(1, 3)$ versus $(3, 1)$. Its aggregate totals coincide while ordered usage laws are disjoint. A noisy example with probabilities $3/4$ versus $1/4$ has total variation $1/2$ and posterior Bayes vulnerability $3/4$ under a uniform binary prior. These synthetic cases show why observer choice and the complete conditional law belong in the contract; they do not estimate a provider.

An additional example samples a hidden first response with the same visible usage. One private arm chooses deterministic second usage from that response; the other uses the same marginal mixture. Both ordered laws agree. Strong bisimulation splits their states; exact span checking proves equality, showing a concrete precision improvement.

4. Impact and validation

The artifact enables reproducible experiments on relational precision, source translation, stochastic usage observations and evidence trust. It improves the project's original conservative checker and provides executable examples for compiler teaching. There is no independently documented external deployment, commercial impact or user-count claim.

Table 1 summarizes distinct evidence families. The 15 source patterns are partial author-created bounded projections of pinned PDL/APPL/LMQL

files, with omissions and mutations recorded. The identical balanced files are compared against immutable baseline revision `a49fcdbd`. These are not complete framework ports or held-out benchmarks. The two deterministic oracle modes repeat the same corpus and are reported separately.

Evidence	Recorded result
Compiler regression	112 tests; ASan/UBSan and examples pass
Source precision	8/15 baseline admissions versus 15/15 current
Bounded stress	13,824 accepted secret comparisons; no request or usage difference
Cyclic graph fixtures	Six accepted proofs replay; eight rejected fixtures have finite witnesses
Bounded source bridge	17/20 exports; 720 differential run pairs agree; three retry exclusions
Actual LangGraph backend	888 runs and 4,528 call-event pairs agree; 24 call-cap prefixes
Actual Python source	30 accepted trace pairs and six rejected value probes agree
Finite stochastic study	11 kernels, 130 analyses, 260 exact independent histogram comparisons
Stochastic closure	Five equality proofs and six inconclusive cases; all 11 independently replay
Exact trace equivalence	12 cases: 6 equality proofs and 6 finite witnesses; all independently replay

Table 1: Offline evidence families; repeated inputs/seeds are not independent applications.

The bounded full-text interpreter shares its generated AST with source rendering but imports no compiler/runtime facts. Of 120 negative stress cases it witnesses 117; three retry mutations remain unknown on the finite grid. The graph value probe shares producer normalization, while structural replay is separately written. Source/graph interpretation shares the source parser, so it is finite translation evidence. The actual source study uses authored restricted Python functions; 20 frontend tests include stale-attestation invalidation.

The quantitative DP agrees with independently parsed no-merge enumeration in all 260 comparisons. Seventeen slices have equal expected usage but positive total variation; all 11 state-cap probes remain unknown. All 130 saved reports independently replay at explicit limits of one million paths, ten million work steps and one GiB byte work. At the default 64 MiB limit,

one largest trace case remains unknown. Producer counters and approximate logarithms are excluded from replayed facts.

The stochastic closure study yields five equality proofs and six inconclusive cases. All 11 certificates replay independently. The proved cases also pass 60 DP checks at horizons 1, 2, 4, 8, 16 and 32 with both observers, covering 120 unit-mass scenario histograms. All 11 fresh state-cap certificates and attestations remain unknown. These checks corroborate implementation behavior. Partition stability supports the result for every finite horizon under the declared model; neither the tests nor replay validate a provider.

The exact trace-law corpus adds the equal-but-non-bisimilar example to those eleven models. All 12 decisions independently replay. Equality cases pass 48 finite DP checks; all 6 finite witness probabilities agree with DP. All 12 fresh cap probes remain unknown, and 12 semantic report mutations reject. The span certificate, rather than sampled horizons, supports all-finite-horizon equality.

5. Conclusions

OrchLang makes the boundary between a token ceiling, request equivalence and usage-law equality explicit in runnable software. Typed request identities, dependent results, productive cycles, source provenance and independent checks give a stronger inspectable artifact than its preceding bounded demo. The contribution is a focused specialization of established relational and probabilistic methods. Provider accounting validation, independently reused applications, mechanized soundness and richer framework extraction remain future work.

Declaration of generative AI and AI-assisted technologies

Codex and GPT-6 Luna assisted implementation, testing, adversarial review, literature investigation and manuscript drafting. This draft records their use. The human author's review, edits and responsibility statement must be completed before submission; this text does not assert that approval has occurred.

Author completion and availability

The author must confirm authorship, affiliation, correspondence, CRediT roles, funding and competing interests. The MIT-licensed repository [12] supplies code, manuals and generated offline evidence. An immutable public release link must be supplied after candidate freeze. No DOI or editorial acceptance is asserted.

Metadata

Candidate metadata; public immutable release and support contact await completion.

Nr.	Code metadata description	Value
C1	Current code version	0.5.0-rc1 (research candidate)
C2	Permanent link to code/repository used for this code version	`https://github.com/guyoverclocked/orchlang-compiler-design-project`; immutable candidate link pending
C3	Permanent link to Reproducible Capsule	No hosted capsule assigned; local reproduction manifest supplied
C4	Legal Code License	MIT
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	C++17; Python; optional LangGraph 1.2.14; HTML/JavaScript demo
C7	Compilation requirements, operating environments and dependencies	C++17, make, Python 3.10+; optional locked LangGraph environment
C8	If available, link to developer documentation/manual	Repository `docs/`: installation, contracts, reproduction, user/developer notes
C9	Support email for questions	To be confirmed by corresponding author

Table 2: Code metadata (mandatory)

References

[1] V.C. Ngo, M. Dehesa-Azuara, M. Fredrikson, J. Hoffmann, Verifying and synthesizing constant-resource implementations with types, IEEE Symposium on Security and Privacy (2017). `https://doi.org/10.1109/SP.2017.53`.

[2] E. Çiçek, G. Barthe, M. Gaboardi, D. Garg, J. Hoffmann, Relational cost analysis, POPL (2017) 316–329. `https://doi.org/10.1145/3009837.3009858`.

[3] L. Mandel, G. Baudart, M. Vaziri, M. Hirzel, PPDL: LLM-based flows as probabilistic programs, ICML, PMLR 306 (2026) 86172–86197. https://proceedings.mlr.press/v306/mandel26a.html.

[4] G. Smith, On the foundations of quantitative information flow, FOSSACS (2009). https://doi.org/10.1007/978-3-642-00596-1_21.

[5] P. Schröer, F. Randone, R. Pardo, A. Wąsowski, Symbolic quantitative information flow for probabilistic programs (2024). https://doi.org/10.1007/978-3-031-75783-9_6.

[6] K.G. Larsen, A. Skou, Bisimulation through probabilistic testing, Information and Computation 94(1) (1991). https://doi.org/10.1016/0890-5401(91)90030-6.

[7] W.-G. Tzeng, A polynomial-time algorithm for the equivalence of probabilistic automata, SIAM Journal on Computing 21(2) (1992) 216–227. https://doi.org/10.1137/0221017.

[8] S. Kiefer, A.S. Murawski, J. Ouaknine, B. Wachter, J. Worrell, Language equivalence for probabilistic automata, CAV, LNCS 6806 (2011) 526–540. https://doi.org/10.1007/978-3-642-22110-1_42.

[9] S. Kiefer, A.S. Murawski, J. Ouaknine, B. Wachter, J. Worrell, On the complexity of equivalence and minimisation for Q-weighted automata, Logical Methods in Computer Science 9(1) (2013), article 8. https://doi.org/10.2168/LMCS-9(1:8)2013.

[10] S. Kiefer, A.S. Murawski, J. Ouaknine, B. Wachter, J. Worrell, APEX: An analyzer for open probabilistic programs, CAV, LNCS 7358 (2012) 693–698. https://doi.org/10.1007/978-3-642-31424-7_51.

[11] LangChain, LangGraph 1.2.14 [software], Python Package Index (2026), accessed 9 October 2026. https://pypi.org/project/langgraph/1.2.14/.

[12] Nambi Rajan M, OrchLang 0.5.0-rc1 [software candidate], GitHub repository (2026), accessed 9 October 2026. https://github.com/guyoverclocked/orchlang-compiler-design-project.